



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Q-OP

REQUIREMENTS ANALYSIS AND SPECIFICATION DOCUMENT
(RASD)
AUTOMATION AND CONTROL ENGINEERING

Group: CW

Team Members:

Özgür Deniz Akbay - ID: 11049499

Mauro Ingenito - ID: 10783278

Nermin Bozkurt - ID: 11048958

Professor: Prof. Matteo Camilli

Tutor: Livia Lestingi

Academic Year: 2025-2026

Contents

Contents	i
1 Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms, Abbreviations	1
1.4 Document Structure	1
2 Overall Description	2
2.1 Scenarios	2
2.2 Domain Model	2
2.3 User Characteristics	4
2.4 Product Functions	4
2.5 Nonfunctional Aspects	4
2.6 Assumptions, Dependencies, and Constraints	5
3 Additional Models	6
3.1 Requirements-Level Sequence Diagrams	6
3.1.1 Routine Booking	6
3.1.2 Machine Lifecycle Management	6
3.1.3 Routine Optimization	7
3.2 Finite State Machines	8
3.2.1 Machine Lifecycle FSM	8
3.2.2 Booking Lifecycle FSM	9
Bibliography	11
A Appendix A	12
List of Figures	13
List of Tables	14

1 | Introduction

1.1. Purpose

The purpose of this project is to apply software engineering practices to develop Q-OP, a web application designed to explicitly manage the usage of specific gym equipment and optimize workout routines.

1.2. Scope

Q-OP addresses the lack of machine-level demand tracking in standard gym booking systems. The system regulates facility access and estimates equipment occupancy by requiring users to book sessions based on their specific workout routines.

1.3. Definitions, Acronyms, Abbreviations

- **RASD:** Requirements Analysis and Specification Document.
- **Routine:** A predefined sequence of gym equipment categories a user intends to use during a session.
- **MachineType:** A category of fitness equipment (e.g., Bench Press, Cable Crossover) characterized by a specific average exercise duration.
- **Maximum Capacity (C_{max}):** The dynamically calculated limit of users a specific *MachineType* can safely support during a given time slot without severe bottlenecks.

1.4. Document Structure

This document outlines the overall description of the system, including scenarios, product functions, and nonfunctional aspects. It also contains the domain model, requirements-level sequence diagrams, and finite state machines.

2 | Overall Description

2.1. Scenarios

Scenario 1: Routine Booking. A user plans to execute a Push/Pull/Legs split. They select their "Push" routine and a Monday 18:00 time slot. The system identifies the required *MachineTypes* (e.g., Bench Press, Cable Crossover) and aggregates the projected demand from all users booking those types during that slot. It calculates the maximum capacity (C_{max}) based on the average exercise durations and the current number of operational physical machines (M). Finding that the expected occupancy remains within the C_{max} threshold, the system displays the expected waiting times and allows the user to confirm the booking.

Scenario 2: Machine Lifecycle Management. A gym owner notices a physical cable machine is damaged. They log into the owner dashboard and update that specific machine's state to "Broken". The system immediately decrements the operational capacity (M) for the "Cable Crossover" *MachineType* and recalculates its C_{max} . It then evaluates all confirmed bookings for upcoming slots. Discovering that the reduced capacity can no longer support the previously confirmed demand, the system automatically invalidates the most recently booked sessions requiring that *MachineType* and sends cancellation and rescheduling notifications to the affected users.

Scenario 3: Routine Optimization. A user attempts to book a routine during a peak hour. The system detects that the aggregated demand for a required *MachineType* exceeds its calculated C_{max} for the selected time slot. To resolve the bottleneck, the system suggests an alternative time slot with lower occupancy, or proposes an optimized execution order for the user's workout routine. If the user accepts, the optimized routine is booked. If the user declines the alternatives, the booking process is aborted, preventing the facility from exceeding its capacity limits.

2.2. Domain Model

This class diagram (fig. 2.1) represents the structural foundation of the system, utilizing standard UML notation to map the core entities and their associations.

To facilitate dynamic capacity calculations, the equipment architecture is split into two entities: *MachineType* (representing the category and average usage duration) and *Machine* (representing the physical hardware and its operational status). Key relationships include the composition between the Gym and Machine entities, alongside the association between Users and WorkoutRoutines, which encapsulates the central business logic of the load-balanced booking process.

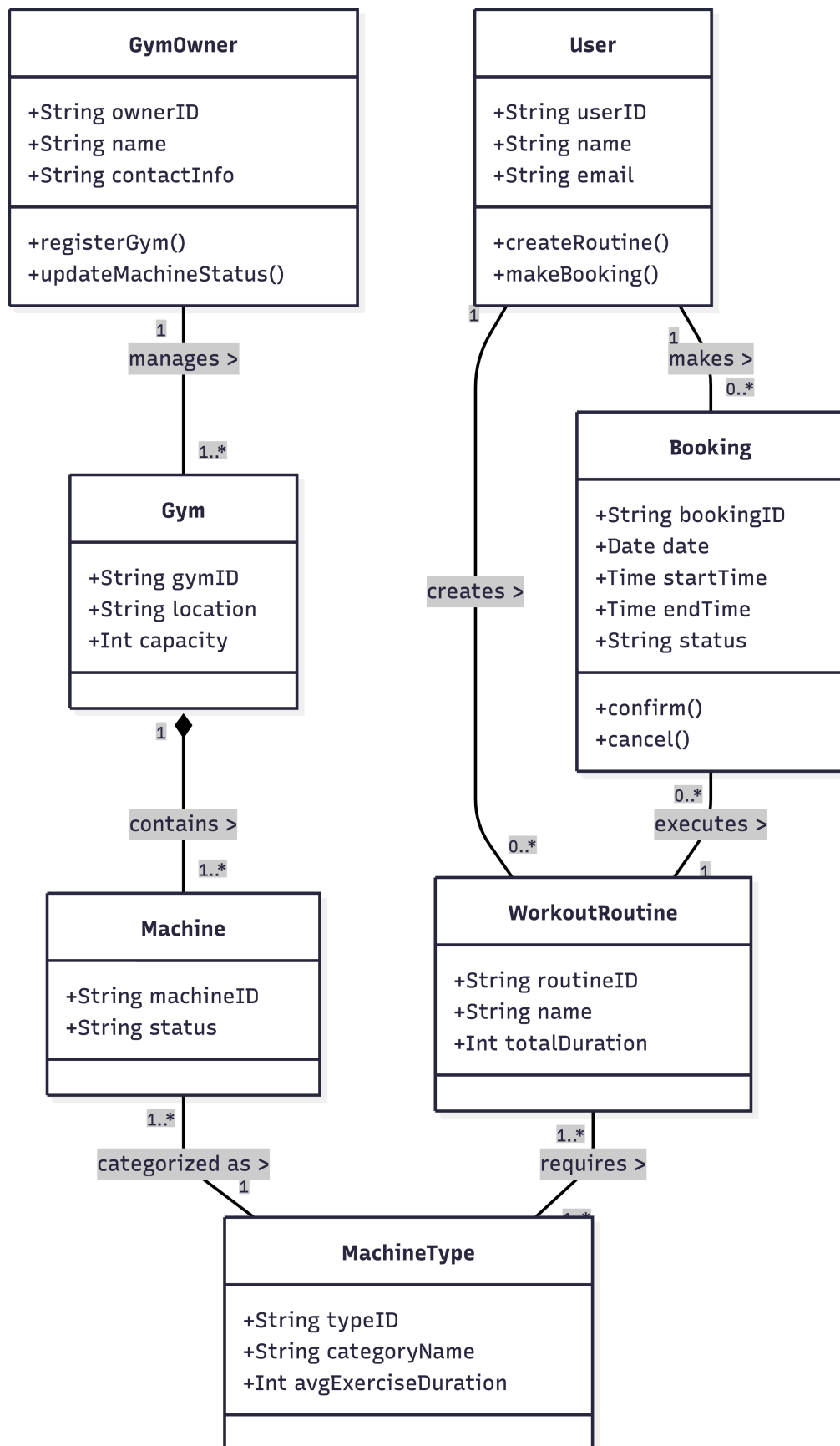


Figure 2.1: Q-OP Domain Model

2.3. User Characteristics

- **Gym Users:** Individuals who follow structured weekly workout routines. They require efficiency and predictability to complete their workouts within limited time windows.
- **Gym Owners:** Facility managers who require a precise view of machine-level demand to make informed decisions regarding equipment maintenance, replacement, and gym capacity.

2.4. Product Functions

- The system shall allow gym owners to register one or more gyms, categorizing physical equipment under specific *MachineTypes*.
- The system shall allow users to create and save custom workout routines defined by required *MachineTypes*.
- The system shall dynamically calculate the maximum user capacity (C_{max}) for a specific *MachineType* during a given time slot using the following formula:

$$C_{max} = \left(\frac{T_{slot}}{t_{exercise}} \right) \times M$$

Where T_{slot} is the total slot duration, $t_{exercise}$ is the average exercise time for that type, and M is the number of physical machines currently in the **Available** state.

- The system shall continuously monitor the aggregate demand for a *MachineType* across all booked routines for a given time slot and display the expected occupancy prior to booking confirmation.
- The system shall dynamically update M and recalculate C_{max} if a gym owner updates a physical machine's lifecycle state (e.g., to **Broken** or **Under Maintenance**).
- The system shall prevent booking confirmations if the aggregated user demand for any required *MachineType* exceeds its current C_{max} .
- The system shall suggest alternative time slots, alternative *MachineTypes*, or an optimized workout order when requested equipment reaches maximum capacity.

2.5. Nonfunctional Aspects

- **Performance:** The system shall compute and return machine availability predictions and waiting-time estimates within 2 seconds of a user selecting a time slot.
- **Availability:** The booking and notification services shall be available 24/7 to accommodate users planning workouts outside of standard gym operating hours.
- **Scalability:** The system architecture must cleanly separate domain data across multiple gyms without data leakage.

2.6. Assumptions, Dependencies, and Constraints

- **Assumption:** Users physically attend the gym during their confirmed time slots and adhere to both the equipment list and the estimated exercise durations defined in their workout routines.
- **Assumption:** Gym owners manually and accurately update the lifecycle states of their machines (e.g., from "available" to "under maintenance").

3 | Additional Models

3.1. Requirements-Level Sequence Diagrams

The following models illustrate the dynamic behavior of the Q-OP system, specifically demonstrating how the system handles aggregated demand and dynamic capacity calculations across the core functional scenarios.

3.1.1. Routine Booking

This diagram (fig. 3.1) models the booking process. Before confirming a reservation, the system queries the database to calculate the maximum capacity (C_{max}) and compares it against the aggregated demand. The model utilizes an *alternative* (*alt*) fragment to demonstrate the user's final decision: confirming the reservation if the predicted wait time is acceptable, or abandoning the request.

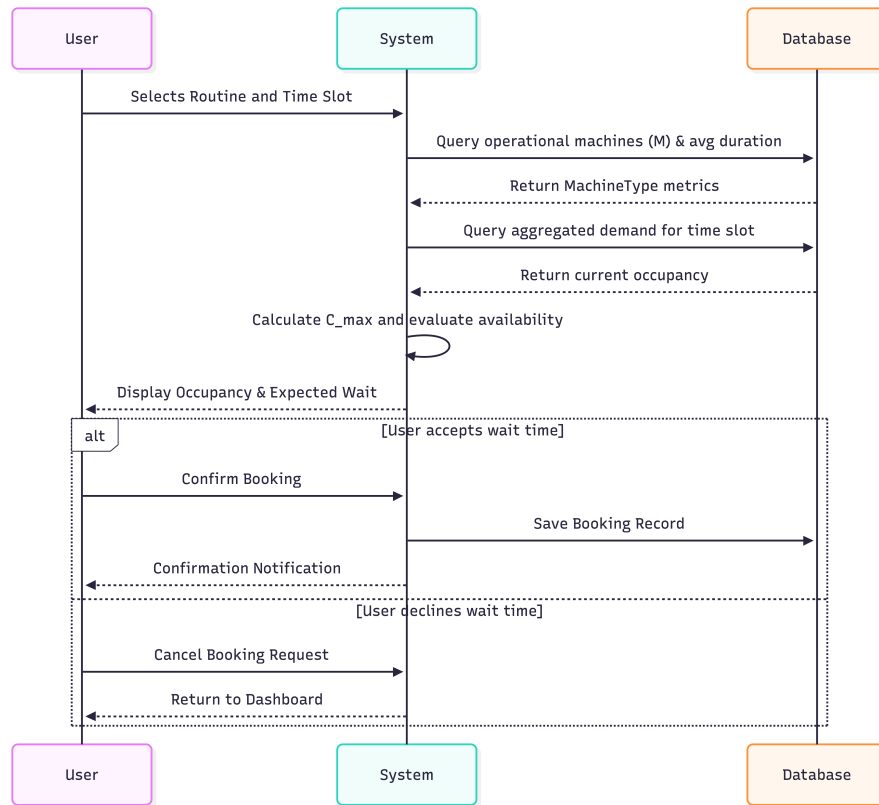


Figure 3.1: Routine Booking Sequence

3.1.2. Machine Lifecycle Management

This interaction (fig. 3.2) shows the automated safety mechanisms triggered when a gym owner marks a physical machine as broken. The system decrements the available instances

(M) and recalculates C_{max} . An *optional* (*opt*) fragment is utilized to evaluate the resulting state: the system will only invalidate bookings and notify users if the remaining physical capacity can no longer support the previously confirmed demand.

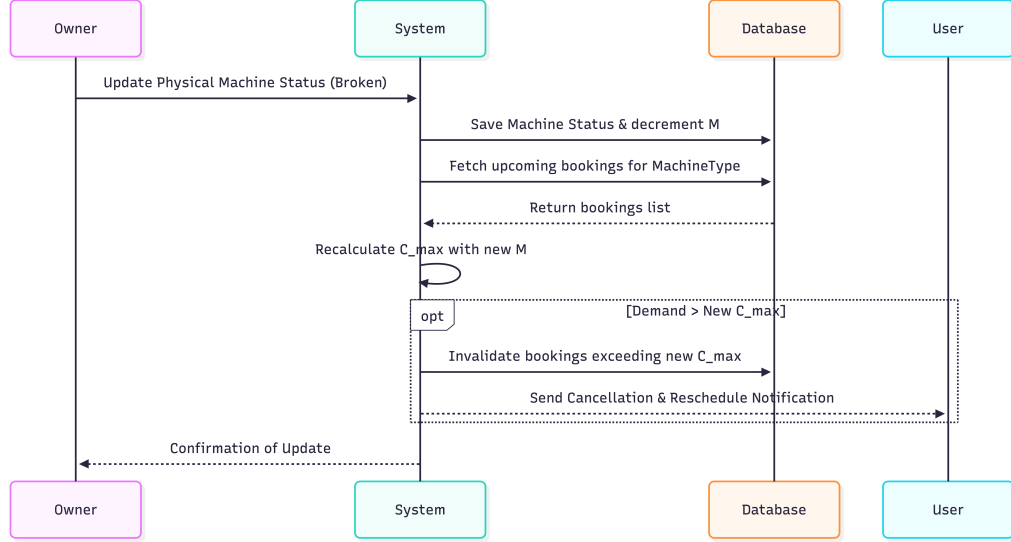


Figure 3.2: Machine Lifecycle Management Sequence

3.1.3. Routine Optimization

This diagram (fig. 3.3) highlights the system's analytical capabilities during peak hours. When the system detects that the aggregated user demand for a specific *MachineType* will exceed its C_{max} , it triggers an optimization algorithm to propose alternatives. The model utilizes an *alternative* (*alt*) fragment to demonstrate both outcomes: the successful database update if the user accepts the suggestion, and the safe termination of the booking process if the user declines.

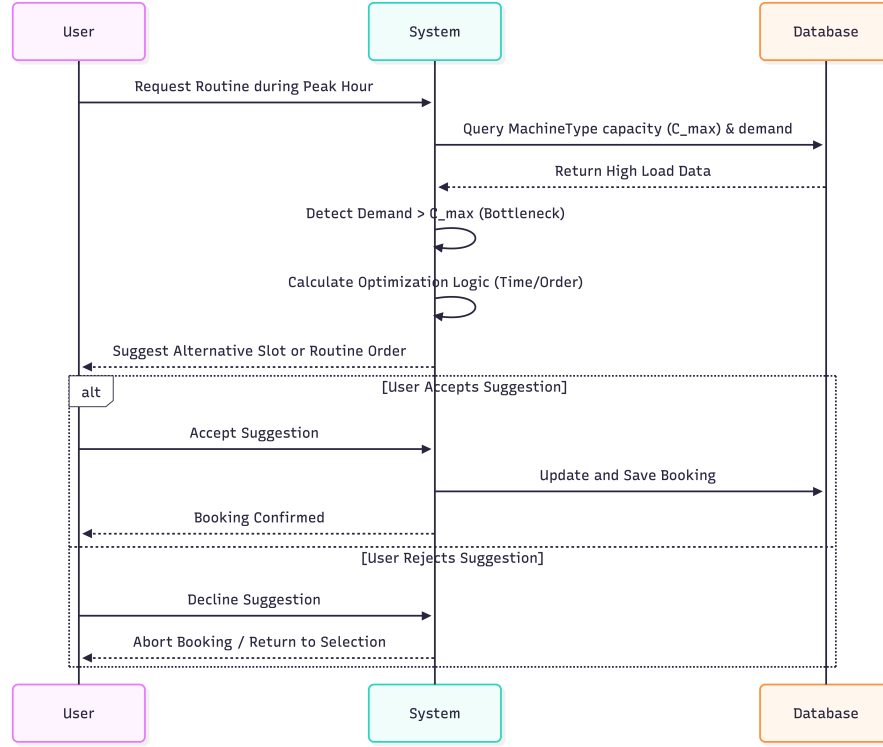


Figure 3.3: Routine Optimization Sequence

3.2. Finite State Machines

This section presents the formal behavioral modeling for the primary operational entities of the Q-OP system: the gym equipment (**Machine**) and the user reservation (**Booking**). These models ensure the system maintains logical consistency across state transitions.

3.2.1. Machine Lifecycle FSM

The Machine Finite State Machine (fig. 3.4) regulates the physical operational status of a specific piece of equipment. Because occupancy and booking validation are calculated dynamically at the *MachineType* level, the physical machine's states are strictly limited to its maintenance lifecycle. Its current state determines whether it contributes to the operational capacity (M) of its corresponding *MachineType*.

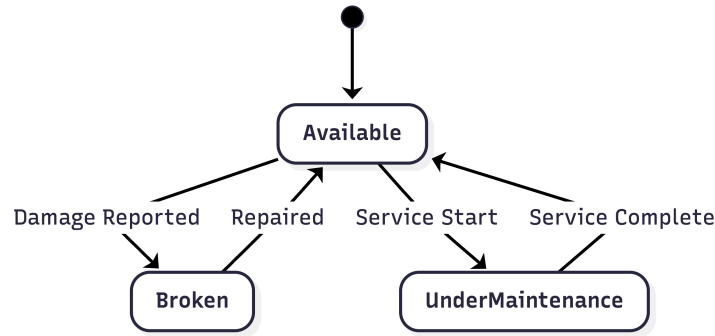


Figure 3.4: Machine Operational Lifecycle FSM

State Descriptions and Transition Logic:

- **Available:** The default state of a machine upon registration. It is fully functional and actively contributes to the total operational capacity (M) of its *MachineType*.
- **Broken:** Safe failure state triggered manually by a gym owner when damage is reported. The machine no longer contributes to M , which lowers the C_{max} threshold and may trigger system-wide booking cancellations. It returns to *Available* only after a successful repair event.
- **Under Maintenance:** A state reserved for scheduled safety checks and preventive servicing. Like the *Broken* state, it temporarily removes the unit from the available capacity pool (M).

3.2.2. Booking Lifecycle FSM

The Booking Finite State Machine (fig. 3.5) governs the validity of a user's reservation from initialization to completion or cancellation.

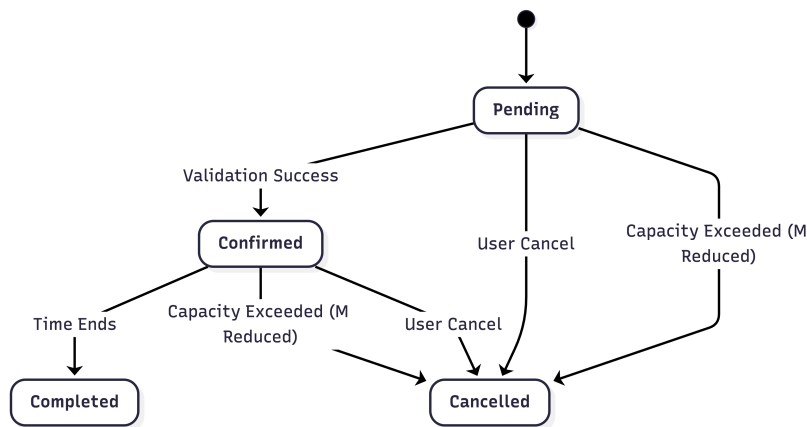


Figure 3.5: User Booking Lifecycle FSM

State Descriptions and Transition Logic:

- **Pending:** The initial state when a user submits a booking request. The system temporarily holds the slot while evaluating aggregate demand against C_{max} .
- **Confirmed:** Transited into once the system successfully validates availability. The slot is officially secured for the user.
- **Completed:** The final successful state reached automatically when the allocated time slot concludes.
- **Cancelled:** A terminal state reached via manual user cancellation, or automatically by the system if physical hardware failures reduce the operational capacity (M) to a point where current demand exceeds the new C_{max} .

Bibliography

A | Appendix A

List of Figures

2.1	Q-OP Domain Model	3
3.1	Routine Booking Sequence	6
3.2	Machine Lifecycle Management Sequence	7
3.3	Routine Optimization Sequence	8
3.4	Machine Operational Lifecycle FSM	9
3.5	User Booking Lifecycle FSM	9

List of Tables